

Informe de Laboratorio 06

Tema: Django Admin

Nota

Estudiantes	Escuela	Asignatura
Fredy José Aragón Carpio Diego Marcelo Arce Coaquira Gustavo Alonso Carrillo Villalta José Manuel Bravo Rojas	Escuela Profesional de Ingeniería de Sistemas	Desarrollo de Aplicaciones Web Semestre: III Código: 2502117 Grupo: Desarrolladores

Laboratorio	Tema	Duración
06	Django Admin	04 horas

Semestre académico	Fecha de inicio	Fecha de entrega
2026 - A	18 de Mayo de 2026	22 de Mayo de 2026

Índice

1. Laboratorio 06: Django Admin	3
1.1. Descripción y Objetivos	3
1.2. Entregables	3
1.3. Entorno de Implementación	3
2. Equipos, materiales y temas utilizados	4
3. URL del Repositorio en GitHub	4
4. Desarrollo de actividades	4
4.1. Estructura del proyecto	4
4.2. Entorno virtual	5
4.2.1. Contenido de requirements.txt	6
4.3. Creación del proyecto y aplicación	6
4.4. Principales Modelos DER	6
4.5. Modelos Principales del Sistema	8

4.5.1. Modelo <code>roles.py</code>	8
4.5.2. Modelo <code>users.py</code>	8
4.5.3. Modelo <code>categories.py</code>	9
4.5.4. Modelo <code>products.py</code>	10
4.5.5. Modelo <code>orders.py</code>	11
4.5.6. Modelo <code>promotions.py</code>	13
4.6. Validadores personalizados	14
4.7. Configuraci�n de Django Admin	15
4.7.1. Panel principal de Django Admin	15
4.7.2. Listado de entidades registradas	16
4.8. Creaci�n del superusuario	16
4.9. Migraciones y base de datos	17
4.9.1. Ejecuci�n autom�tica del superusuario	18
4.10. Proceso completo en Django Admin	18
4.10.1. Creaci�n de registros (CREATE)	18
4.10.2. Lectura de registros (READ)	19
4.10.3. Actualizaci�n de registros (UPDATE)	19
4.10.4. Eliminaci�n de registros (DELETE)	20
5. Resultados	21
6. Autoevaluaci�n	24
7. R�brica de Calificaci�n	24
8. Referencias	24

1. Laboratorio 06: Django Admin

Django Admin es la interfaz administrativa incorporada en Django para gestionar datos de manera centralizada y segura. En este laboratorio se implement  el registro de modelos, la configuraci n de `ModelAdmin`, el uso de validadores, la redefinici n de `save()` y `__str__()` en modelos independientes, adem s de las capturas del proceso completo de CRUD.

La pr ctica estuvo orientada a preparar un proyecto Django con una estructura organizada, usar un entorno virtual, crear una aplicaci n con nombre siglado, y dejar evidencia del funcionamiento de Django Admin con un modelo de datos resumido de 4 a 7 tablas.

1.1. Descripci n y Objetivos

En esta pr ctica se busc  reforzar el uso de Django Admin como herramienta de gesti n de datos, apoy ndose en modelos bien definidos y restricciones desde el propio modelo. Tambi n se trabaj  con entorno virtual, archivo `requirements.txt`, creaci n de proyecto y app, y configuraci n de CRUDs autom ticos.

- Crear y activar un entorno virtual para aislar dependencias del proyecto.
- Generar el archivo `requirements.txt` con las librer as necesarias.
- Crear el proyecto Django y la aplicaci n correspondiente.
- Dise ar modelos en archivos independientes siguiendo el est ndar solicitado.
- Implementar el m todo `save()` para l gica previa al guardado.
- Implementar el m todo `__str__()` para mostrar registros legibles en Django Admin.
- Aplicar validadores en campos de los modelos.
- Registrar los modelos en `admin.py` y personalizar `ModelAdmin`.
- Ejecutar migraciones y verificar la base de datos.
- Documentar el proceso con capturas de los CRUDs en Django Admin.

1.2. Entregables

Los entregables del laboratorio se organizan en el repositorio, el video y el informe final. El video, cuando se solicite, debe publicarse en una plataforma accesible p blicamente y compartirse mediante URL directa.

- **Repositorio:** <https://github.com/FredyAragon/Cafe-Sys>
- **Video (Versi n larga):** <https://youtu.be/i-UfTTAAYN8>
- **Video (Versi n corta):** <https://youtu.be/nrhBVw-aWWAR>
- **Informe:** <https://github.com/FredyAragon/Cafe-Sys/blob/main/informes/Informe-Lab6-DAW.pdf>

1.3. Entorno de Implementaci n

Para el desarrollo se utiliz  Python con Django, un entorno virtual para la instalaci n aislada de dependencias, editor VS Code, Docker, Docker Compose y una base de datos PostgreSQL. El backend se ejecut  dentro de un contenedor construido desde un `Dockerfile`, mientras que la base de datos se levant  con la imagen oficial de PostgreSQL.

- Python 3.10 o superior.
- Django.
- Entorno virtual `venv`.

- Docker y Docker Compose.
- PostgreSQL 17.
- Visual Studio Code.
- Git y GitHub.

2. Equipos, materiales y temas utilizados

- Sistema Operativo: Windows 11.
- Python.
- Django.
- Entorno virtual `venv`.
- Docker.
- Docker Compose.
- Visual Studio Code.
- PostgreSQL.
- Git y GitHub.
- Navegador web para pruebas de Django Admin.
- Herramienta para diagramas ER.

3. URL del Repositorio en GitHub

El proyecto desarrollado en este laboratorio se encuentra documentado en GitHub junto con el informe PDF. Los enlaces quedan declarados para completar en la versión final del informe.

- **Repositorio:** <https://github.com/FredyAragon/Cafe-Sys>
- **Informe PDF:** <https://github.com/FredyAragon/Cafe-Sys/blob/main/informes/Informe-Lab6-DAW.pdf>

4. Desarrollo de actividades

4.1. Estructura del proyecto

La estructura del proyecto Django generado sigue una organización estándar con el proyecto principal, la aplicación del dominio, archivos de configuración y carpeta de plantillas o imágenes si corresponde.

```
Cafe-Sys/  
  
|----backend/  
    |----config/  
        |----__init__.py  
        |----asgi.py  
        |----settings.py  
        |----urls.py  
        +----wsgi.py
```

```
|----api/
|----migrations/
|----models/
|----__init__.py
|----categories.py
|----deliveries.py
|----drivers.py
|----inventories.py
|----locations.py
|----messages.py
|----order_details.py
|----orders.py
|----products.py
|----products_promotions.py
|----promotions.py
|----reviews.py
|----roles.py
|----users.py
|----validators.py
+----vehicles.py

|----admin.py
|----serializers.py
|----views.py
|----urls.py
|----apps.py
+----tests.py

|----.env
|----create_superuser.py
|----Dockerfile
|----manage.py
|----requirements.txt
+----README.md

|----bd/
|----cafesys.sql
|----DER.png
+----README.md

|----frontend/
+----README.md

|----informes/
|----img/
+----Informe-Lab06-DAW.pdf

|----docker-compose.yml
|----.gitignore
+----README.md
```

4.2. Entorno virtual

El entorno virtual se crea para aislar las dependencias del proyecto y evitar conflictos con otras instalaciones de Python. El archivo `requirements.txt` debe incluir al menos Django, psycpg2 y

cualquier librería adicional usada en el laboratorio.

Listing 1: Creación y activación del entorno virtual.

```
python -m venv venv
venv\Scripts\activate
pip install django psycopg2-binary
pip freeze > requirements.txt
pip install -r requirements.txt
```

4.2.1. Contenido de requirements.txt

```
asgiref==3.11.1
Django==6.0.5
django-rest-framework==3.17.1
psycopg2-binary==2.9.12
python-decouple==3.8
sqlparse==0.5.5
tzdata==2026.2
```

4.3. Creación del proyecto y aplicación

En esta parte se generó el proyecto principal y la aplicación del dominio. Luego, la app se registró en `INSTALLED_APPS` del archivo `settings.py`.

Listing 2: Creación del proyecto y la aplicación.

```
django-admin startproject project .
python manage.py startapp app
```

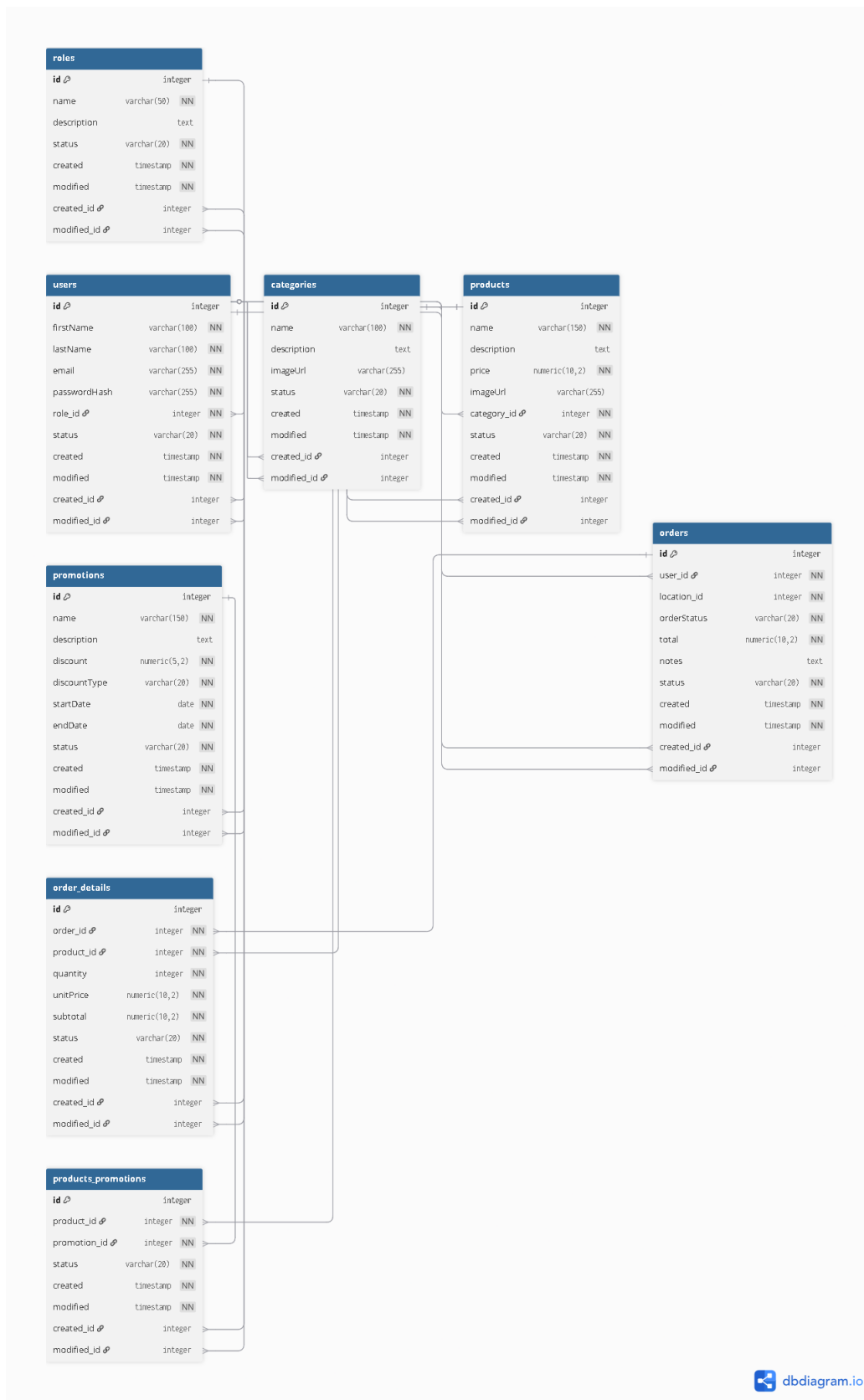
Listing 3: Registro de la aplicación en settings.py.

```
# Application definition

INSTALLED_APPS = [
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',
    'rest_framework',
    'api',
]
```

4.4. Principales Modelos DER

El modelo entidad-relación resume la estructura de datos usada en el proyecto final. Se recomienda un rango de 4 a 7 tablas para evidenciar relaciones claras y permitir la demostración de CRUDs en Django Admin.



4.5. Modelos Principales del Sistema

Cada modelo fundamental del sistema fue definido en un archivo independiente dentro del paquete de la aplicación para mejorar la mantenibilidad y escalabilidad del proyecto. Los modelos incluyen de forma transversal los atributos obligatorios de auditoría e integridad de datos: **id** (clave primaria), **status**, **created**, **modified**, así como las relaciones exógenas de seguimiento **created_id** y **modified_id**. La validación de las reglas de negocio se centralizó mediante el método nativo **clean()** junto a validadores de campo personalizados, garantizando la persistencia de datos limpios en el motor relacional.

4.5.1. Modelo roles.py

Listing 4: Modelo Roles.

```
from django.db import models
from .validators import validate_not_blank

class Roles(models.Model):
    # ID principal explícito requerido por el DER
    id = models.AutoField(primary_key=True)

    name = models.CharField(max_length=50, unique=True, validators=[
        validate_not_blank])
    description = models.TextField(blank=True, null=True)
    status = models.CharField(max_length=20, default='active',
        choices=[('active', 'Active'), ('inactive', 'Inactive')])
    created = models.DateTimeField(auto_now_add=True)
    modified = models.DateTimeField(auto_now=True)

    # Campos de auditoría forzados para la BD según el DER
    created_id = models.ForeignKey('api.Users', on_delete=models.SET_NULL, null=True, blank=True,
        related_name='roles_created', db_column='created_id')
    modified_id = models.ForeignKey('api.Users', on_delete=models.SET_NULL, null=True,
        blank=True, related_name='roles_modified', db_column='modified_id')

    class Meta:
        db_table = 'roles'
        verbose_name = 'Role'
        verbose_name_plural = 'Roles'

    def save(self, *args, **kwargs):
        # Asegurar la ejecución segura de los validadores de atributos individuales
        self.clean_fields()

        # Normalizar nombre a minúsculas
        if self.name:
            self.name = self.name.strip().lower()

        super().save(*args, **kwargs)

    def __str__(self):
        return self.name
```

4.5.2. Modelo users.py

Listing 5: Modelo Roles.

```
from django.db import models
from django.core.validators import MinLengthValidator
from .validators import validate_not_blank

class Users(models.Model):
    # 1. ID principal explícito requerido por la rbrica y tu DER
    id = models.AutoField(primary_key=True)

    firstName = models.CharField(max_length=100, validators=[validate_not_blank])
    lastName = models.CharField(max_length=100, validators=[validate_not_blank])
    email = models.EmailField(unique=True)
    passwordHash = models.CharField(max_length=255, validators=[MinLengthValidator(8)])

    # 2. Relación corregida (Nombre limpio en Python, columna exacta del DER en la BD)
    role = models.ForeignKey('api.Roles', on_delete=models.PROTECT, related_name='
        users', db_column='role_id')

    status = models.CharField(max_length=20, default='active',
        choices=[('active', 'Active'), ('inactive', 'Inactive')])
    created = models.DateTimeField(auto_now_add=True)
    modified = models.DateTimeField(auto_now=True)

    # 3. Campos de auditoría forzados para la BD según el DER
    created_id = models.ForeignKey('api.Users', on_delete=models.SET_NULL, null=True, blank
        =True, related_name='users_created', db_column='created_id')
    modified_id = models.ForeignKey('api.Users', on_delete=models.SET_NULL, null=True,
        blank=True, related_name='users_modified', db_column='modified_id')

    class Meta:
        db_table = 'users'
        verbose_name = 'User'
        verbose_name_plural = 'Users'

    def save(self, *args, **kwargs):
        # Asegurar que se ejecuten las restricciones de los atributos individuales
        self.clean_fields()

        # Normalizar email a minúsculas antes de guardar
        if self.email:
            self.email = self.email.lower().strip()

        # Normalizar nombre con capitalización
        if self.firstName:
            self.firstName = self.firstName.strip().title()
        if self.lastName:
            self.lastName = self.lastName.strip().title()

        super().save(*args, **kwargs)

    def __str__(self):
        return f"{self.firstName} {self.lastName} ({self.email})"
```

4.5.3. Modelo categories.py

Listing 6: Modelo Roles.

```
from django.db import models
from .validators import validate_not_blank, validate_http_url

class Categories(models.Model):
    # 1. ID principal explícito como pide la recomendación
    id = models.AutoField(primary_key=True)

    name = models.CharField(max_length=100, unique=True, validators=[
        validate_not_blank])
    description = models.TextField(blank=True, null=True)
    imageUrl = models.CharField(max_length=255, blank=True, null=True, validators=[
        validate_http_url])
    status = models.CharField(max_length=20, default='active',
        choices=[('active', 'Active'), ('inactive', 'Inactive')])
    created = models.DateTimeField(auto_now_add=True)
    modified = models.DateTimeField(auto_now=True)

    # 2. Forzar el nombre de la columna en la BD para cumplir con la rbrica y evitar el "
    # _id_id"
    created_id = models.ForeignKey('api.Users', on_delete=models.SET_NULL, null=True, blank
        =True, related_name='categories_created', db_column='created_id')
    modified_id = models.ForeignKey('api.Users', on_delete=models.SET_NULL, null=True,
        blank=True, related_name='categories_modified', db_column='modified_id')

    class Meta:
        db_table = 'categories'
        verbose_name = 'Category'
        verbose_name_plural = 'Categories'

    def save(self, *args, **kwargs):
        # CORRECCIN: Ejecuta los validadores personalizados de tus atributos sin romper el
        # flujo del Admin
        self.clean_fields()

        # Normalizar nombre con capitalización
        if self.name:
            self.name = self.name.strip().title()

        super().save(*args, **kwargs)

    def __str__(self):
        return self.name
```

4.5.4. Modelo products.py

Listing 7: Modelo Roles.

```
from django.db import models
from django.core.validators import MinValueValidator
from .validators import validate_not_blank, validate_http_url

class Products(models.Model):
```

```
# 1. ID principal explícito requerido por la rbrica y tu DER
id = models.AutoField(primary_key=True)

name = models.CharField(max_length=150, validators=[validate_not_blank])
description = models.TextField(blank=True, null=True)
price = models.DecimalField(max_digits=10, decimal_places=2, validators=[
    MinValueValidator(0.01)])
imageUrl = models.CharField(max_length=255, blank=True, null=True, validators=[
    validate_http_url])

# 2. Relación corregida (Nombre limpio en Python, columna exacta del DER en la BD)
category = models.ForeignKey('api.Categories', on_delete=models.PROTECT, related_name
    ='products', db_column='category_id')

status = models.CharField(max_length=20, default='active',
    choices=[('active', 'Active'), ('inactive', 'Inactive')])
created = models.DateTimeField(auto_now_add=True)
modified = models.DateTimeField(auto_now=True)

# 3. Campos de auditoría forzados para la BD según el DER
created_id = models.ForeignKey('api.Users', on_delete=models.SET_NULL, null=True, blank
    =True, related_name='products_created', db_column='created_id')
modified_id = models.ForeignKey('api.Users', on_delete=models.SET_NULL, null=True,
    blank=True, related_name='products_modified', db_column='modified_id')

class Meta:
    db_table = 'products'
    verbose_name = 'Product'
    verbose_name_plural = 'Products'

def save(self, *args, **kwargs):
    # Asegurar que se ejecuten las restricciones de los atributos individuales
    self.clean_fields()

    # Normalizar nombre con capitalización
    if self.name:
        self.name = self.name.strip().title()

    # Redondear precio a 2 decimales
    if self.price is not None:
        self.price = round(self.price, 2)

    super().save(*args, **kwargs)

def __str__(self):
    return f"{self.name} - S/. {self.price}"
```

4.5.5. Modelo orders.py

Listing 8: Modelo Roles.

```
from django.db import models
from django.core.validators import MinValueValidator
```

```
class Orders(models.Model):
    ORDER_STATUS = [
        ('pending', 'Pending'),
        ('preparing', 'Preparing'),
        ('ready', 'Ready'),
        ('delivered', 'Delivered'),
        ('cancelled', 'Cancelled'),
    ]

    # 1. ID principal explícito requerido por la rbrica y tu DER
    id = models.AutoField(primary_key=True)

    # 2. Relaciones corregidas (Nombre limpio en Python, columnas exactas del DER en la BD)
    user = models.ForeignKey('api.Users', on_delete=models.CASCADE, related_name='
        orders', db_column='user_id')
    location = models.ForeignKey('api.Locations', on_delete=models.PROTECT, related_name='
        orders', db_column='location_id')

    orderStatus = models.CharField(max_length=20, choices=ORDER_STATUS, default='pending')
    total = models.DecimalField(max_digits=10, decimal_places=2, validators=[
        MinValueValidator(0)])
    notes = models.TextField(blank=True, null=True)
    status = models.CharField(max_length=20, default='active',
        choices=[('active', 'Active'), ('inactive', 'Inactive')])
    created = models.DateTimeField(auto_now_add=True)
    modified = models.DateTimeField(auto_now=True)

    # 3. Campos de auditora forzados para la BD segn el DER
    created_id = models.ForeignKey('api.Users', on_delete=models.SET_NULL, null=True, blank
        =True, related_name='orders_created', db_column='created_id')
    modified_id = models.ForeignKey('api.Users', on_delete=models.SET_NULL, null=True,
        blank=True, related_name='orders_modified', db_column='modified_id')

    class Meta:
        db_table = 'orders'
        verbose_name = 'Order'
        verbose_name_plural = 'Orders'

    def save(self, *args, **kwargs):
        # Asegurar que se ejecuten las restricciones del validador de total
        self.clean_fields()

        # Si el pedido es cancelado, marcarlo como inactivo
        if self.orderStatus == 'cancelled':
            self.status = 'inactive'

        # Redondear total
        if self.total is not None:
            self.total = round(self.total, 2)

        super().save(*args, **kwargs)

    def __str__(self):
        user_display = f"{self.user.firstName} {self.user.lastName}" if self.user else "No
            User"
        return f"Order #{self.id} | {user_display} | {self.orderStatus} | S/. {self.total}"
```

4.5.6. Modelo promotions.py

Listing 9: Modelo Roles.

```
from django.db import models
from django.core.validators import MinValueValidator, MaxValueValidator
from django.utils import timezone
from .validators import validate_not_blank, validate_end_date

class Promotions(models.Model):
    DISCOUNT_TYPES = [
        ('percentage', 'Percentage'),
        ('fixed', 'Fixed Amount'),
    ]

    # 1. ID principal explícito requerido por la rbrica y tu DER
    id = models.AutoField(primary_key=True)

    name = models.CharField(max_length=150, validators=[validate_not_blank])
    description = models.TextField(blank=True, null=True)
    discount = models.DecimalField(max_digits=5, decimal_places=2,
                                   validators=[MinValueValidator(0.01), MaxValueValidator(100)])
    discountType = models.CharField(max_length=20, choices=DISCOUNT_TYPES)
    startDate = models.DateField()
    endDate = models.DateField()
    status = models.CharField(max_length=20, default='active',
                              choices=[('active', 'Active'), ('inactive', 'Inactive')])
    created = models.DateTimeField(auto_now_add=True)
    modified = models.DateTimeField(auto_now=True)

    # 2. Campos de auditora forzados para la BD segn el DER
    created_id = models.ForeignKey('api.Users', on_delete=models.SET_NULL, null=True, blank=True,
                                   related_name='promotions_created', db_column='created_id')
    modified_id = models.ForeignKey('api.Users', on_delete=models.SET_NULL, null=True,
                                    blank=True, related_name='promotions_modified', db_column='modified_id')

    class Meta:
        db_table = 'promotions'
        verbose_name = 'Promotion'
        verbose_name_plural = 'Promotions'

    def save(self, *args, **kwargs):
        # Ejecutar restricciones de atributos individuales
        self.clean_fields()
        # Ejecutar la validacin personalizada de fechas (validate_end_date)
        self.clean()

        # Si la fecha de fin ya pas, desactivar automticamente
        if self.endDate and self.endDate < timezone.now().date():
            self.status = 'inactive'

        super().save(*args, **kwargs)

    def clean(self):
        # Llama a tu funcin que valida que la fecha de fin sea posterior a la de inicio
        validate_end_date(self)
```

```
def __str__(self):  
    return f"{self.name} - {self.discount}{'%' if self.discountType == 'percentage'  
        else ' S/.'}"
```

4.6. Validadores personalizados

Con el objetivo de reforzar la integridad de los datos y evitar registros inconsistentes desde Django Admin o desde cualquier punto de acceso al modelo, se implementaron validadores personalizados en el archivo `validators.py`. Estos validadores permiten centralizar reglas de negocio reutilizables y mantener los modelos m s organizados.

Los validadores implementados permiten:

- Evitar campos vac os o con  nicamente espacios en blanco.
- Verificar que las URLs comiencen con `http` o `https`.
- Validar que la fecha de fin de una promoci n sea posterior o igual a la fecha de inicio.
- Validar coherencia entre fechas de salida y entrega en entregas de pedidos.

Listing 10: Archivo `validators.py`.

```
from django.core.exceptions import ValidationError  
  
def validate_not_blank(value):  
    if not value or not value.strip():  
        raise ValidationError(  
            'This field cannot be blank or whitespace.'  
        )  
  
def validate_http_url(value):  
    if value and not value.startswith('http'):  
        raise ValidationError(  
            'URL must start with http or https.'  
        )  
  
def validate_end_date(instance):  
    if instance.endDate and instance.startDate:  
        if instance.endDate < instance.startDate:  
            raise ValidationError(  
                'End date must be greater than or equal to start date.'  
            )  
  
def validate_delivery_dates(instance):  
    if instance.deliveredAt and instance.departureAt:  
        if instance.deliveredAt < instance.departureAt:  
            raise ValidationError(  
                'Delivered date cannot be before departure date.'  
            )
```

Estos validadores fueron utilizados directamente en los modelos mediante el atributo `validators` y tambi n desde m todos personalizados como `clean()` y `save()`, permitiendo asegurar restricciones antes de almacenar informaci n en la base de datos.

4.7. Configuraci n de Django Admin

Los modelos fueron registrados en el archivo `admin.py` para habilitar autom ticamente las operaciones CRUD desde Django Admin. Esto permiti  administrar registros mediante una interfaz web segura y centralizada.

El registro de modelos se realiz  utilizando `admin.site.register()`, permitiendo que cada entidad aparezca autom ticamente en el panel administrativo.

Listing 11: Registro de modelos en `admin.py`.

```
from django.contrib import admin
from .models import *

admin.site.register(Roles)
admin.site.register(Users)
admin.site.register(Categories)
admin.site.register(Products)
admin.site.register(Inventories)
admin.site.register(Promotions)
admin.site.register(ProductsPromotions)
admin.site.register(Orders)
admin.site.register(OrderDetails)
admin.site.register(Locations)
admin.site.register(Deliveries)
admin.site.register(Drivers)
admin.site.register(Vehicles)
admin.site.register(Reviews)
admin.site.register(Messages)
```

La configuraci n realizada permiti  visualizar las entidades registradas, acceder a formularios autom ticos, aplicar filtros y ejecutar operaciones de creaci n, edici n y eliminaci n directamente desde el navegador.

4.7.1. Panel principal de Django Admin

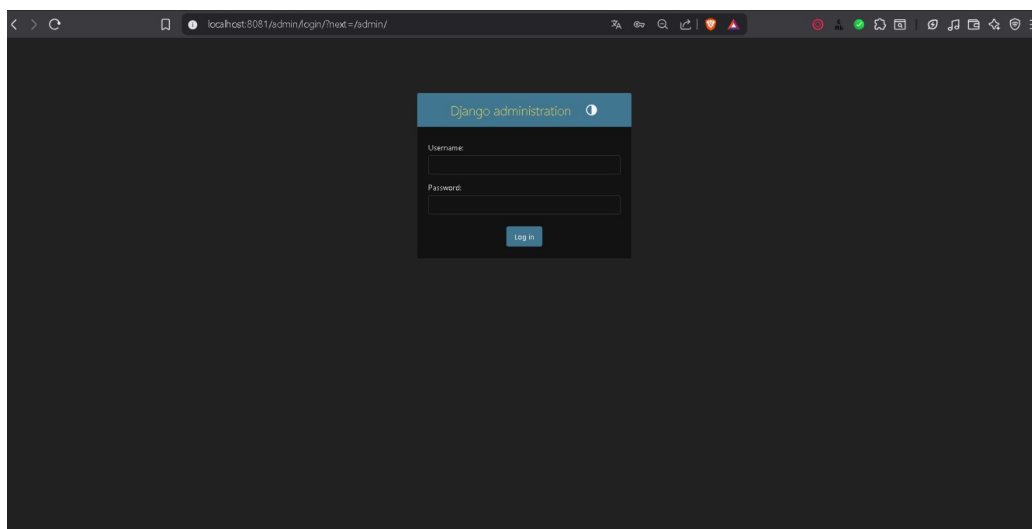


Figura 2: Vista principal del panel administrativo de Django.

4.7.2. Listado de entidades registradas

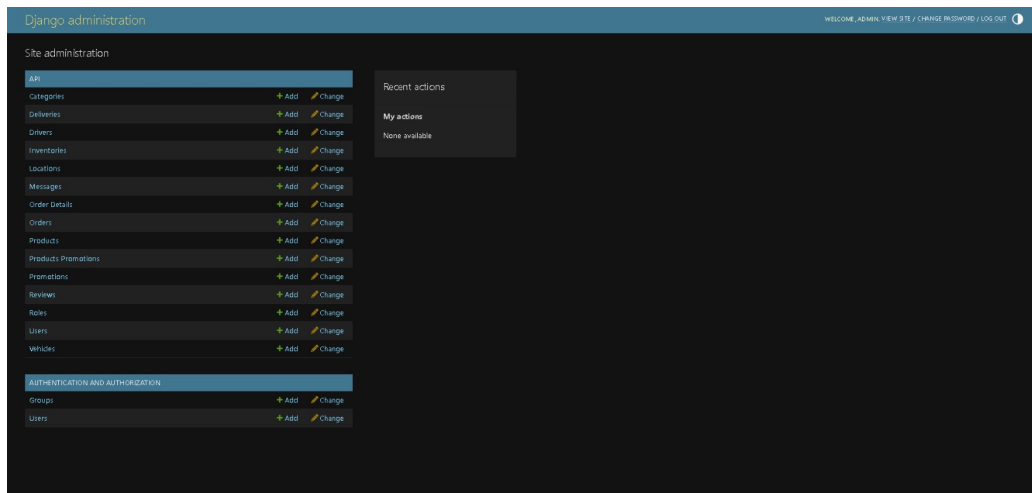


Figura 3: Modelos registrados dentro de Django Admin.

4.8. Creaci n del superusuario

Para acceder al panel administrativo de Django fue necesario crear un superusuario con privilegios completos sobre el sistema. En lugar de utilizar  nicamente el comando manual de Django, se implement  un script automatizado denominado `create_superuser.py`, ejecutado autom ticamente desde Docker Compose al iniciar el contenedor backend.

Este enfoque permiti  automatizar la creaci n del usuario administrador y evitar repetir el proceso manualmente en cada despliegue del entorno.

Listing 12: Script `create_superuser.py`.

```
import os
import django

os.environ.setdefault(
    'DJANGO_SETTINGS_MODULE',
    'config.settings'
)

django.setup()

from django.contrib.auth import get_user_model

User = get_user_model()

if not User.objects.filter(username='admin').exists():
    User.objects.create_superuser(
        'admin',
        'admin@cafesys.com',
        'admin1234'
    )

print(
    'Superusuario creado: admin / admin1234'
```

```
)  
else:  
    print('Superusuario ya existe.')
```

Luego de ejecutar las migraciones y levantar los contenedores, el sistema generó automáticamente el usuario administrador con credenciales predeterminadas para acceder al panel /admin.

Listing 13: Acceso al panel administrativo.

```
http://localhost:8081/admin
```

4.9. Migraciones y base de datos

Para la ejecución del proyecto se utilizó Docker y Docker Compose, con un contenedor para PostgreSQL y otro para el backend Django. El servicio de base de datos se definió a partir de la imagen oficial de PostgreSQL, mientras que el servicio backend se construyó desde el Dockerfile ubicado en la carpeta **Backend**. Además, se expuso el puerto 8081 para acceder al servidor Django desde el host.

El archivo **docker-compose.yml** fue configurado para esperar que PostgreSQL esté disponible antes de iniciar el backend, usando **healthcheck** y **depends_on** con la condición **service_healthy**. De esta manera, se evita que Django intente ejecutar migraciones antes de que la base de datos esté lista.

Listing 14: Servicio backend y base de datos en docker-compose.yml.

```
services:  
  
  db:  
    image: postgres:17  
    container_name: cafesys_db  
    environment:  
      POSTGRES_DB: cafeteriadb  
      POSTGRES_USER: postgres  
      POSTGRES_PASSWORD: 1234  
    ports:  
      - "8085:5432"  
    volumes:  
      - postgres_data:/var/lib/postgresql/data  
    healthcheck:  
      test: ["CMD-SHELL", "pg_isready -U postgres"]  
      interval: 5s  
      timeout: 5s  
      retries: 5  
      start_period: 10s  
  
  backend:  
    build:  
      context: ./backend  
    container_name: cafesys_backend  
  
    command: >  
      sh -c "python manage.py migrate &&  
             python create_superuser.py &&  
             python manage.py runserver 0.0.0.0:8081"  
  
    volumes:  
      - ./backend:/app  
    ports:
```

```
- "8081:8081"
depends_on:
  db:
    condition: service_healthy
env_file:
  - ./backend/.env

volumes:
  postgres_data:
```

En el contenedor del backend, primero se ejecutan las migraciones con `python manage.py migrate`. Luego se ejecuta un script personalizado para crear autom ticamente el superusuario, y finalmente se inicia el servidor con `runserver` escuchando en `0.0.0.0:8081`.

Listing 15: Migraciones de Django.

```
python manage.py makemigrations
python manage.py migrate
```

4.9.1. Ejecuci n autom tica del superusuario

4.10. Proceso completo en Django Admin

En esta secci n se documenta el flujo completo de operaciones CRUD realizado desde Django Admin. Las capturas permiten evidenciar el funcionamiento del sistema y la correcta integraci n de los modelos con la interfaz administrativa de Django.

4.10.1. Creaci n de registros (CREATE)

En esta parte se muestran las capturas relacionadas con la creaci n de registros desde formularios generados autom ticamente por Django Admin.

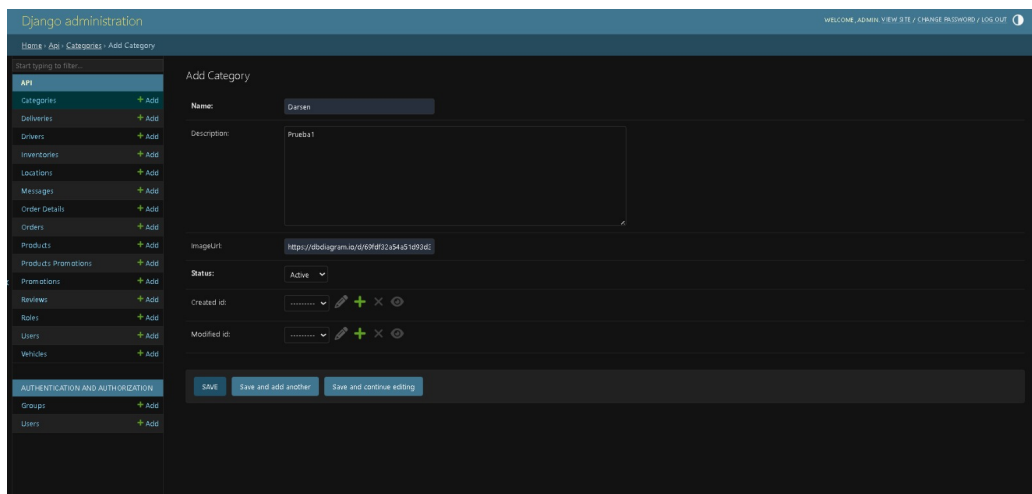


Figura 4: Formulario de creaci n de registros.

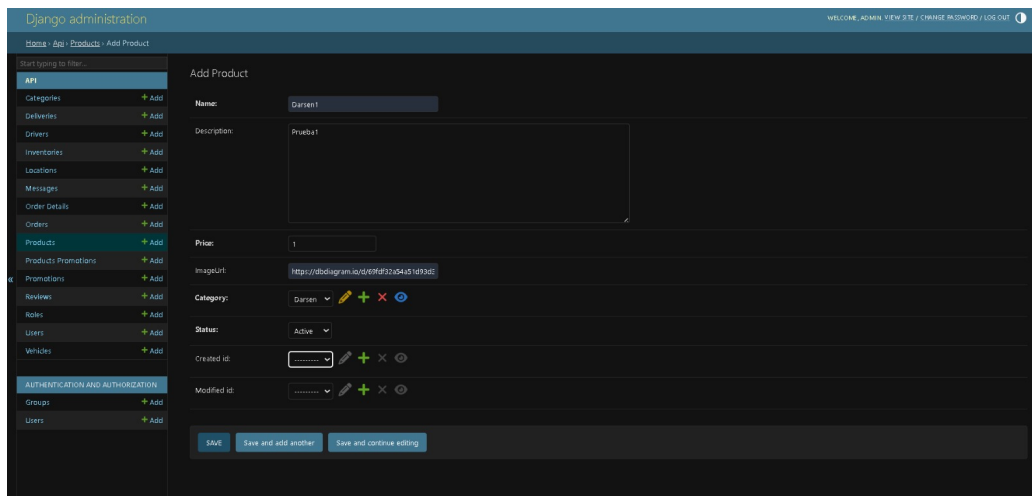


Figura 5: Registro creado correctamente en Django Admin.

4.10.2. Lectura de registros (READ)

Se evidencia el listado autom tico de registros generado por Django Admin, permitiendo b squeda y visualizaci n organizada de la informaci n.

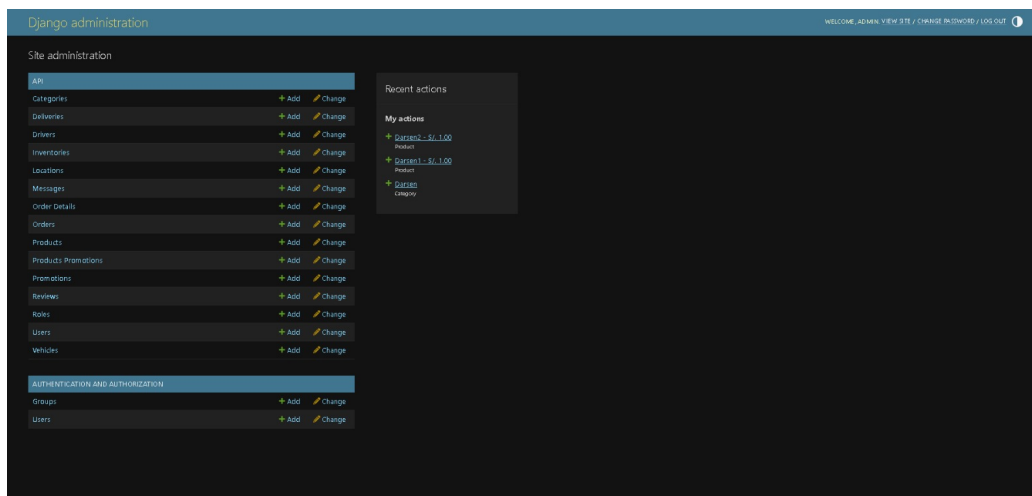
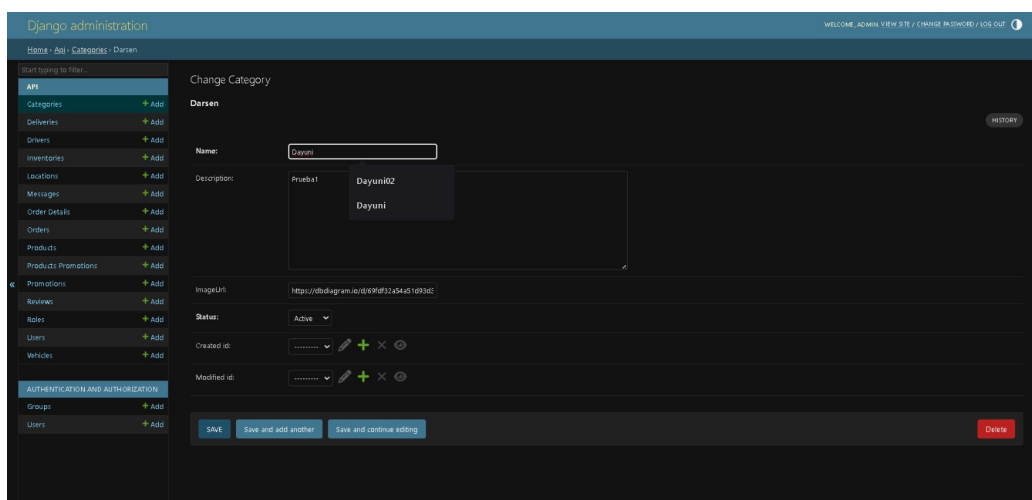


Figura 6: Listado de registros en Django Admin.

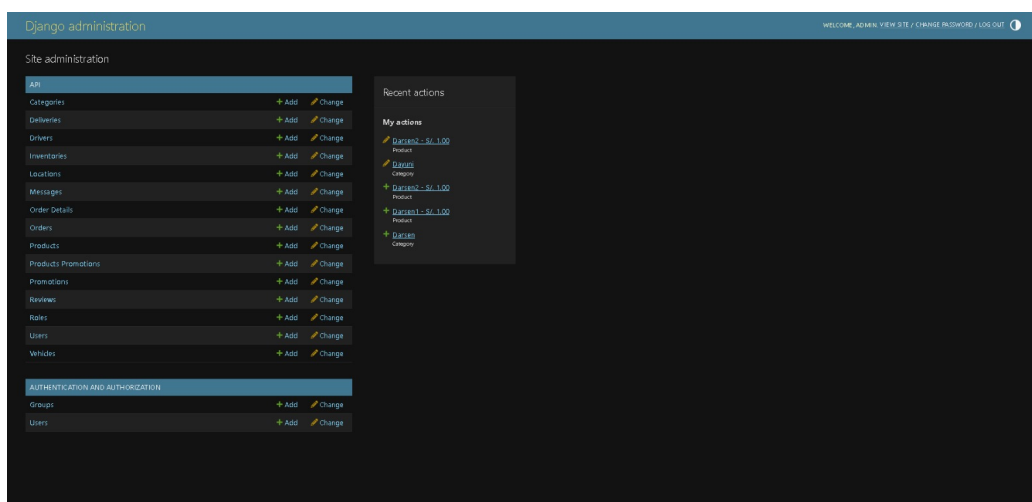
4.10.3. Actualizaci n de registros (UPDATE)

En esta secci n se presentan las capturas relacionadas con la edici n de registros existentes desde la interfaz administrativa.



The screenshot shows the Django Admin interface for editing a 'Darsen' record. The left sidebar contains a menu with categories like API, Categories, Deliveries, Drivers, Inventories, Locations, Messages, Order Details, Orders, Products, Products Promotions, Promotions, Reviews, Roles, Users, and Vehicles. The main form area is titled 'Change Category' and shows the 'Darsen' record. The 'Name' field is 'Darsen'. The 'Description' field contains a list of items: 'Prueba1', 'Dayuni02', and 'Dayuni'. The 'ImageURL' field contains 'https://dbdiagram.io/d/69f73245d1d93dc'. The 'Status' field is set to 'Active'. The 'Created id' and 'Modified id' fields are empty. At the bottom, there are buttons for 'Save', 'Save and add another', 'Save and continue editing', and 'Done'.

Figura 7: Proceso de edici n de registros.



The screenshot shows the Django Admin 'Site administration' page. The left sidebar contains a menu with categories like API, Categories, Deliveries, Drivers, Inventories, Locations, Messages, Order Details, Orders, Products, Products Promotions, Promotions, Reviews, Roles, Users, and Vehicles. The main area displays a list of records for each category, with columns for 'Add', 'Change', and 'Delete'. The 'Recent actions' sidebar on the right shows a list of actions performed, including 'Darsen02 - 5/1/2020', 'Darsen01 - 5/1/2020', 'Darsen03 - 5/1/2020', 'Darsen04 - 5/1/2020', and 'Darsen05 - 5/1/2020'.

Figura 8: Confirmaci n de actualizaci n correcta.

4.10.4. Eliminaci n de registros (DELETE)

Finalmente, se muestra el flujo de eliminaci n de registros utilizando las funcionalidades nativas de Django Admin.

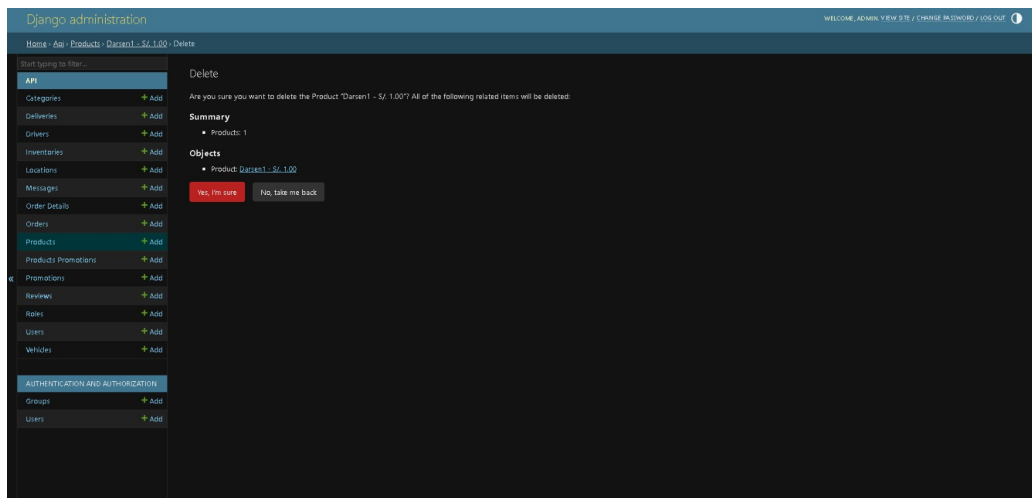


Figura 9: Confirmaci n de eliminaci n de registros.

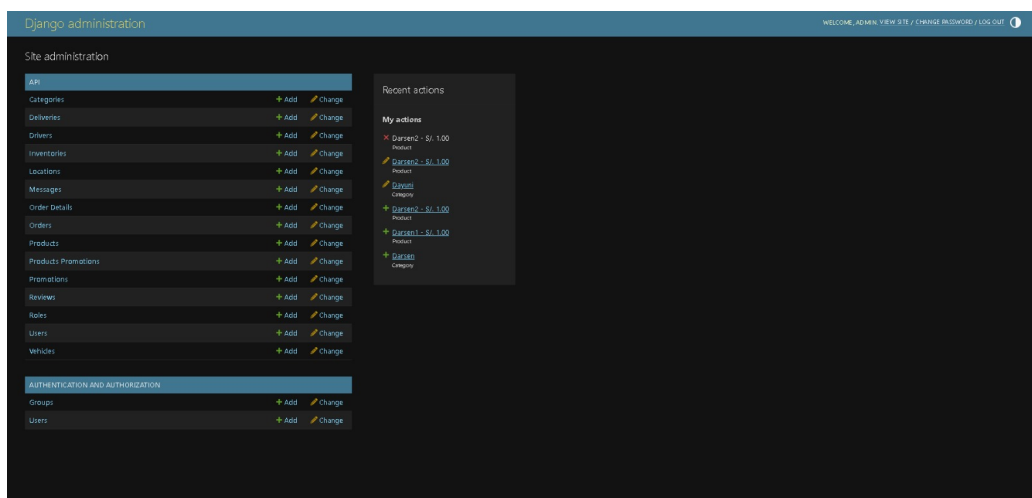


Figura 10: Resultado posterior a la eliminaci n.

5. Resultados

Los resultados evidencian que los modelos fueron creados correctamente, las migraciones se aplicaron sin errores y Django Admin permiti  ejecutar operaciones CRUD sobre cada entidad. Adem s, el diagrama DER final coincide con la estructura implementada en el proyecto.

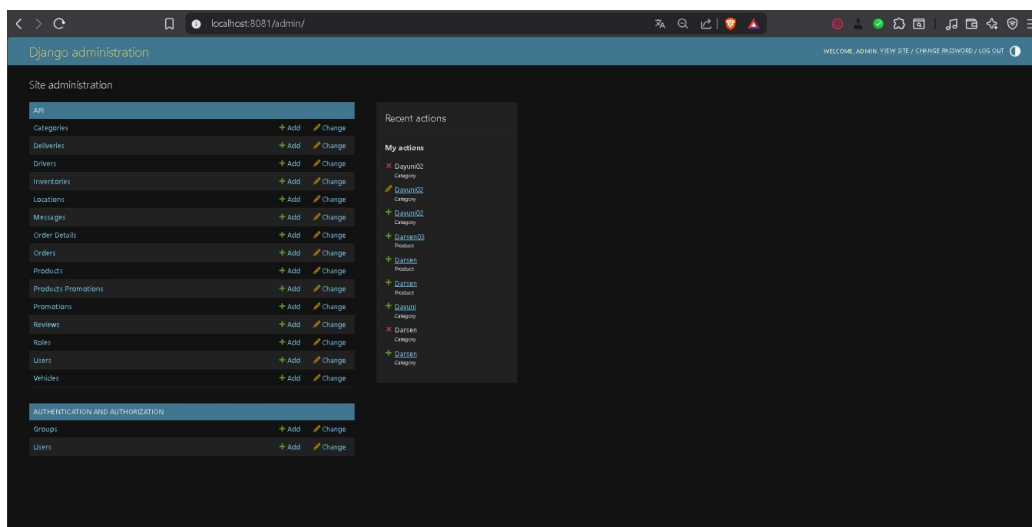


Figura 11: Panel final de Django Admin con los modelos registrados.



6. Autoevaluación

Nombre	Rol	Porcentaje
Fredy José Aragón Carpio	Creación de modelos	100 %
Diego Marcelo Arce Coaquira	Elaboración del informe	100 %
Gustavo Alonso Carrillo Villalta	Revisión de evidencias	100 %
José Manuel Bravo Rojas	Organización del repositorio	100 %
Total		100 %

Cuadro 1: Autoevaluación del laboratorio.

7. Rúbrica de Calificación

Ítem	Descripción	Puntaje
Entorno Virtual	Uso correcto del entorno virtual en proyectos de Python.	2
Django Admin	Realiza detalladamente los pasos para crear el administrador de Django a partir de los modelos.	3
Restricciones	Crear restricciones de forma asertiva a partir de los modelos de Django.	3
Modelo DER	Utiliza una herramienta para crear el modelo entidad-relación en Django.	3
BD	Envía la base de datos con los registros existentes para su revisión.	3
Modelos	Realiza las configuraciones necesarias en los modelos para cumplir con los estándares y obtener las funcionalidades solicitadas.	3
Informe	El laboratorio tiene un README.md que detalla toda la práctica.	3
Total		20

Cuadro 2: Rúbrica de evaluación del laboratorio.

8. Referencias

- https://developer.mozilla.org/es/docs/Learn_web_development/Extensions/Server-side/Django/Models
- https://www.w3schools.com/django/django_models.php
- <https://docs.djangoproject.com/en/6.0/topics/db/models/>
- <https://docs.djangoproject.com/en/6.0/ref/models/instances/#validating-objects>

- <https://www.wplogout.com/export-database-diagrams-erd-from-django/>
- <https://github.com/richarteq/django-enrollments>